lab 01

Date : 18/02/2026

Day - wednesday

```
#part A- control flow
marks = [30,40,45,65,67]
valid = []
for m in marks:
  if 0 <= m <= 100:
    valid.append(m)
print("valid marks:",valid)
```

```
valid marks: [30, 40, 45, 65, 67]
```

program 2 ; categorize student

```
marks = 82
if marks >= 75:
  print("distiniction")
elif marks >= 60:
    print("first class")
elif marks >= 40:
      print("pass")
else:
      print("fail")
```

```
distiniction
```

program A3:

```
marks = [35, 50, 60, 20, 80]
count = 0
for m in marks:
 if m >= 40:
  count += 1
print("Number of Passed Students:", count)
```

```
Number of Passed Students: 3
```

PART B : Functions

Program B1: Pass/Fail Function

```
def check_pass(marks):
 if marks >= 40:
  return 1
  return 0
print(check_pass(55))
```

```
1
```

Program B2: Normalize Feature

```
def normalize(value, min_val, max_val):
  return (value - min_val) / (max_val - min_val)
print(normalize(75, 0, 100))
```

```
0.75
```

Program B3: Calculate Mean

```
def calculate_mean(data):
  return sum(data) / len(data)
print(calculate_mean([45, 55, 65]))
```

```
55.0
```

program B4: Attendance elegibility

```
def eligible(attendance):
    return attendance >= 75
print(eligible(80))
```

```
True
```

Part C – File I/O

Program C1: Read File

```
file = open("marks.txt","r")
data = file.readlines()
file.close()
print(data)
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipykernel_30303/2753362059.py in <cell line: 0>()
----> 1 file = open("marks.txt","r")
      2 data = file.readlines()
      3 file.close()
      4 print(data)

FileNotFoundError: [Errno 2] No such file or directory: 'marks.txt'
```

```
file = open("output.txt", "w")
file.write("Data Processed Successfully")
file.close()
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import csv
with open("student_data.csv", "r") as file:
 reader = csv.reader(file)
for row in reader:
  print(row)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

lab 02

Date - 25/02/2026

day - Wednesday

Dataset Used

```
import numpy as np
student_data = np.array([
[85, 78, 92],
[60, 65, 70],
[95, 88, 90],
[72, 75, 68]
])
```

Part A – Array Creation & Structure

```
print("Student Data:\n", student_data)
print("Shape:", student_data.shape)
print("Number of Students:", student_data.shape[0])
print("Number of Subjects:", student_data.shape[1])
print("First Student Record:", student_data[0])
#ml revaleance as feature matrix
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_30303/1868571050.py in <cell line: 0>()
----> 1 print("Student Data:\n", student_data)
      2 print("Shape:", student_data.shape)
      3 print("Number of Students:", student_data.shape[0])
      4 print("Number of Subjects:", student_data.shape[1])
      5 print("First Student Record:", student_data[0])

NameError: name 'student_data' is not defined
```

Part B : vactorised mathode

B1: Add Grace Marks

```
grace_added = student_data + 5
print("After Adding Grace Marks:\n", grace_added)
```

B2: Increase Marks by 10%

```
sclaed_marks = student_data * 1.1
print("After Scaling Marks:\n", sclaed_marks)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_30303/3498080064.py in <cell line: 0>()
----> 1 sclaed_marks = student_data * 1.1
      2 print("After Scaling Marks:\n", sclaed_marks)

NameError: name 'student_data' is not defined
```

B3: Logical Filtering

```
high_scores = student_data[student_data[:,0]>80]
print("Students with Marks > 80:\n", high_scores)
```

B4: Broadcasting Example

```
subject_mean = np.mean(student_data, axis=0)
normalized = student_data - subject_mean
print("Mean-Centered Data:\n", normalized)
```

Part C – Statistical Analysis

C1: Mean per Student

```
avg_per_student = np.mean(student_data, axis=1)
print("Average Marks per Student:\n", avg_per_student)
```

C2: Mean per Subject

```
avg_per_subject = np.mean(student_data, axis=0)
print("Average Marks per Subject:\n", avg_per_subject)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_30303/1279799462.py in <cell line: 0>()
----> 1 avg_per_subject = np.mean(student_data, axis=0)
      2 print("Average Marks per Subject:\n", avg_per_subject)

NameError: name 'np' is not defined
```

C3: Standard Deviation per Subject

```
std_per_subject = np.std(student_data, axis=0)
print("Standard Deviation per Subject:", std_per_subject)
```

C4: Identify Subject with Highest Varience

```
variance_per_subject = np.var(student_data, axis=0)
print("Variance per Subject:", variance_per_subject)
print("Subject with Highest Variance:",
np.argmax(variance_per_subject)+ 1)
```

loop vs vectorisation

```
import time
size = 1000000
data_list = list(range(size))
data_array = np.array(data_list)

start = time.time()
result_loop = []
for i in data_list:
    result_loop.append(i * 2)
loop_time = time.time() - start

start = time.time()
result_vector = data_array * 2
vector_time = time.time() - start
print("Loop Time:", loop_time)
print("Vectorized Time:", vector_time)
print("Speed Improvement Factor:", loop_time / vector_time)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_30303/3493994868.py in <cell line: 0>()
      2 size = 1000000
      3 data_list = list(range(size))
----> 4 data_array = np.array(data_list)
      5
      6 start = time.time()

NameError: name 'np' is not defined
```

Date - 11/03/2026

Lab 03

day - wednesday

Dataset Used

```
import pandas as pd

data = {
    "name":["A","B","C","D"],
    "Study_Hours":[5,3,7,4],
    "Attendance":[80,None,90,70],
    "Marks":[75,55,None,62]

}
df = pd.DataFrame(data)
df.to_csv("student_data.csv", index=False)
```

Part A – Loading and Exploring DataFrames

A1: Load Dataset

```
import pandas as pd
df = pd.read_csv("student_data.csv")
print(df)
```

incpect dataset structure

```
print(df.head())
print(df.info())
print(df.shape)
```

```
   Study_Hours  Attendance  Assignment_Score Participation Having_Laptop  \
0          4.0        34.0               2.0        Medium            No
1          9.0        79.0              10.0        Medium           Yes
2          6.0        80.0               5.0        Medium           Yes
3          3.0        53.0               5.0          High           Yes
4          3.0        67.0               0.0          High           Yes

   CGPA  Final_Marks Final_Result
0  6.95           62         Pass
1  8.42          100         Pass
2   NaN           81         Pass
3  3.79           54         Pass
4  7.16           53         Pass
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Study_Hours       4757 non-null   float64
 1   Attendance        4773 non-null   float64
 2   Assignment_Score  4761 non-null   float64
 3   Participation     4746 non-null   object
 4   Having_Laptop     4758 non-null   object
 5   CGPA              4757 non-null   float64
 6   Final_Marks       5000 non-null   int64
 7   Final_Result      5000 non-null   object
dtypes: float64(4), int64(1), object(3)
memory usage: 312.6+ KB
None
(5000, 8)
```

A3: Column Selection

```
print(df["Marks"])
print(df[["Study_Hours","Marks"]])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'Marks'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                  2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'Marks'
```

part B- data cleaning

B1: Detect Missing Value

```
print(df.isnull())
```

B2: Count Missing Values

```
print(df.isnull().sum())
```

B3: Remove Missing Values

```
df_clean = df.dropna()
print(df_clean)
```

B4: Replace Missing values

```
df["Marks"].fillna(df["Marks"].mean(), inplace=True)
```

```
print(df)
```

Part C – Filtering and Aggregation

C1: Filter High-Scoring Students

```
high_scores = df[df["Marks"] > 70]
print(high_scores)
```

C2: Filter Using Multiple Conditions

```
eligible = df[(df["Marks"] > 60) & (df["Attendance"] > 75)]
print(eligible)
```

C3: Calculate Mean Marks

```
print(df["Marks"].mean())
```

C4: Group-Based Aggregation

```
group_avg = df.groupby("Attendance")["Marks"].mean()
print(group_avg)
```

lab 04

Date - 18/03/26

day - wednesday

Part A : Data Implemantaion

Data set used

```
import pandas as pd

data = {
    "Study_Hours":[5,2,7,3,6],
    "Attendance":[80,50,90,60,85],
    "Assignment_Score":[8,4,9,5,8],
```

```
    "Participation_score":["High","Low","High","Medium","High"],
    "Previous_Marks":[72,45,85,55,78],
    "Final_Result":["Pass","Fail","Pass","Fail","Pass"],
    "Final_Marks":[80,48,88,58,82]
}
df = pd.DataFrame(data)
print(df)
```

Part B – Data encoding

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Participation_Encoded"] = le.fit_transform(df["Participation_score"])
print(df[["Participation_score","Participation_Encoded"]])
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_features = scaler.fit_transform(df[["Study_Hours","Attendance","Assignment_Score","Previous_Marks"]])
print(scaled_features)
```

```
X_class = df[["Study_Hours","Attendance","Assignment_Score","Participation_Encoded","Previous_Marks"]]
y_class = df["Final_Result"]
print(X_class)
print(y_class)
```

```
X_reg = df[["Study_Hours","Attendance","Assignment_Score","Participation_Encoded","Previous_Marks"]]
y_reg = df["Final_Marks"]
print(X_reg)
print(y_reg)
```

LAB : 05

date : 25/03/2026

day : wednesday

PPP

LAB : 06

DATE : 01/04/2026

DAY : WEDNESDAY

Eda using statical and visual methods

```
import pandas as pd
url = "https://raw.githubusercontent.com/anshuman-scse/Final_marks_result_prediction/refs/heads/main/Final_marks_result_predict
df = pd.read_csv(url)
X = df.drop(["Final_Marks","Final_Result"], axis=1)
y_reg = df["Final_Marks"]
y_class = df["Final_Result"]
```

```
print(df.head())
print(X.head())
```

```
   Study_Hours  Attendance  Assignment_Score Participation Having_Laptop  \
0          4.0        34.0               2.0        Medium            No
1          9.0        79.0              10.0        Medium           Yes
2          6.0        80.0               5.0        Medium           Yes
3          3.0        53.0               5.0          High           Yes
4          3.0        67.0               0.0          High           Yes

   CGPA  Final_Marks Final_Result
0  6.95           62         Pass
1  8.42          100         Pass
2   NaN           81         Pass
3  3.79           54         Pass
4  7.16           53         Pass
```

	Study_Hours	Attendance	Assignment_Score	Participation	Having_Laptop	CGPA
0	4.0	34.0	2.0	Medium	No	6.95
1	9.0	79.0	10.0	Medium	Yes	8.42
2	6.0	80.0	5.0	Medium	Yes	NaN
3	3.0	53.0	5.0	High	Yes	3.79
4	3.0	67.0	0.0	High	Yes	7.16

part B - Missing Values

```
print(X.isnull().sum())
```

```
Study_Hours         243
Attendance          227
Assignment_Score    239
Participation       254
Having_Laptop       242
CGPA                243
dtype: int64
```

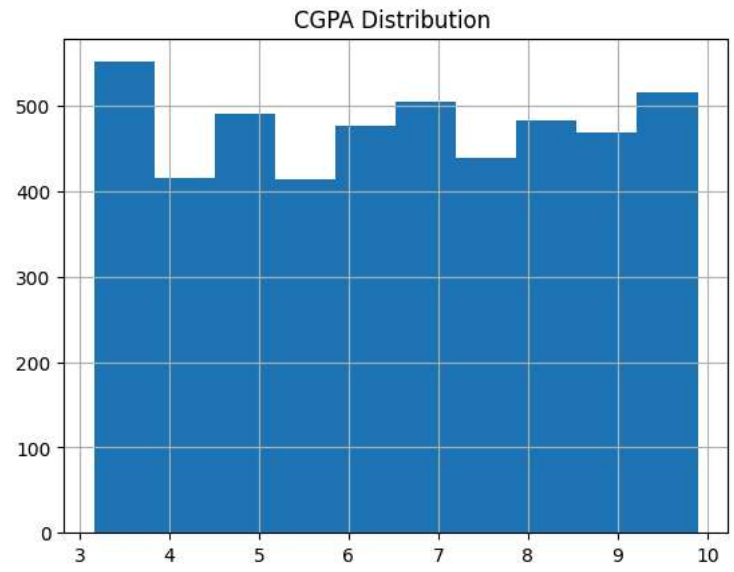
Part c - Statistical Analysis

```
print(X.describe())
```

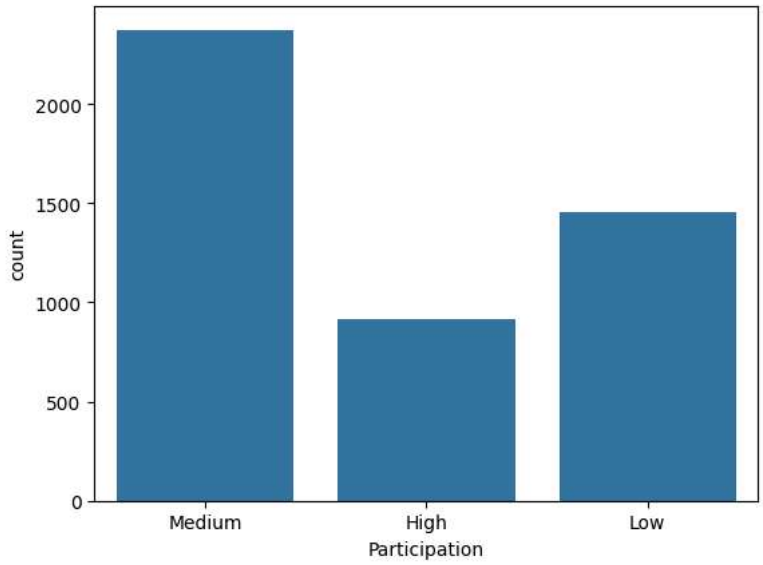
```
       Study_Hours   Attendance  Assignment_Score         CGPA
count  4757.000000  4773.000000       4761.000000  4757.000000
mean      5.478873    64.116279          4.970384     6.524564
std       3.446855    20.139926          3.183229     1.988398
min       0.000000    30.000000          0.000000     3.160000
25%       3.000000    47.000000          2.000000     4.740000
50%       6.000000    64.000000          5.000000     6.530000
75%       8.000000    82.000000          8.000000     8.210000
max      11.000000    99.000000         10.000000     9.890000
```

Part D – Univariate Analysis

```
import matplotlib.pyplot as plt
import seaborn as sns
X["CGPA"].hist()
plt.title("CGPA Distribution ")
plt.show()
```

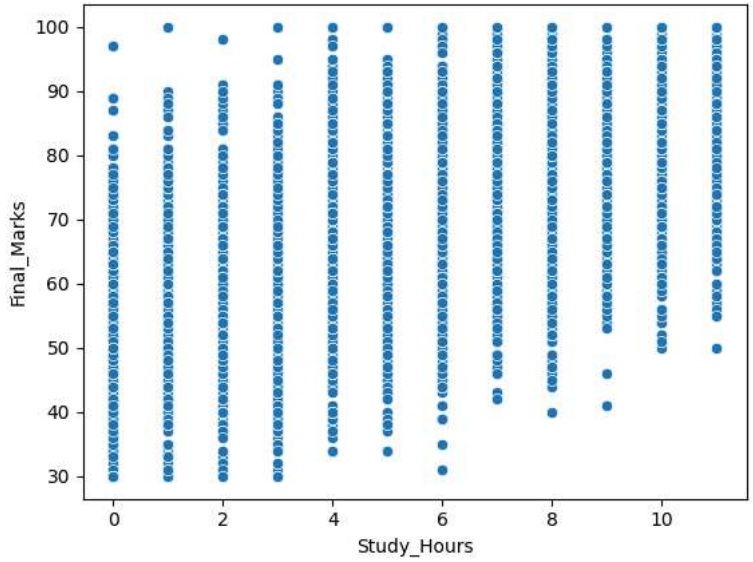


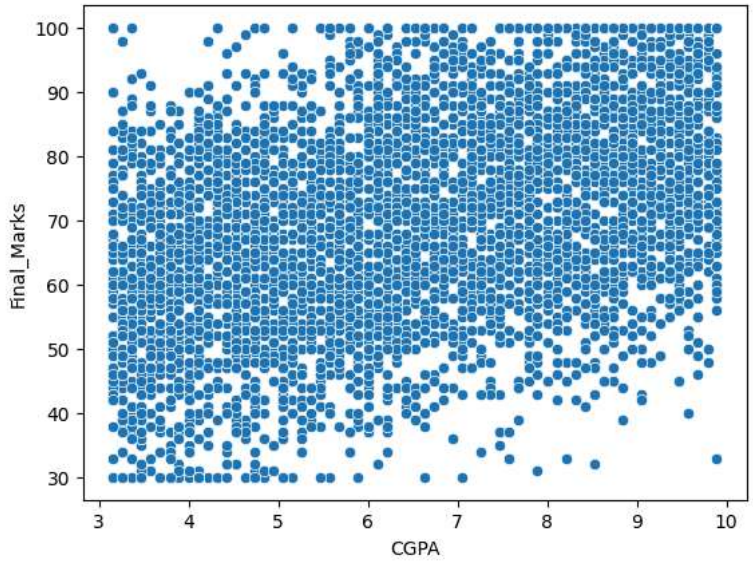

```
sns.countplot(x="Participation",data = X)
plt.show()
```

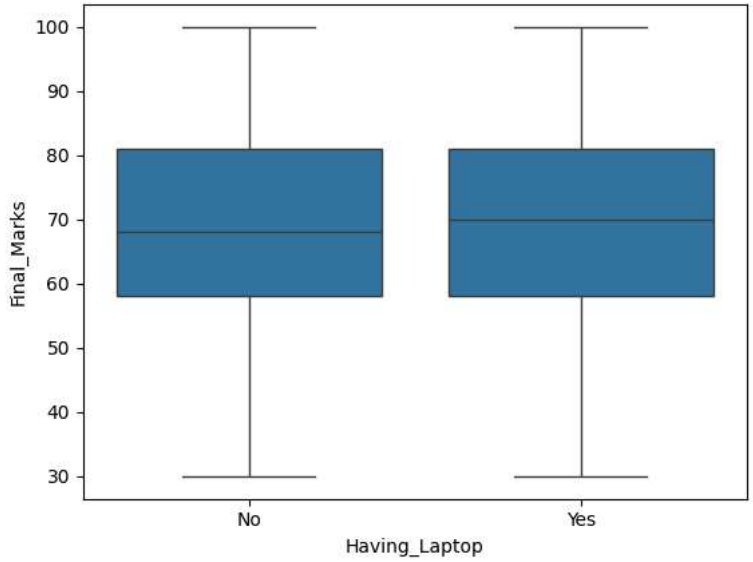


Part E – Feature vs Target Analysis / Bivariate Analysis

```
sns.scatterplot(x=X["Study_Hours"], y=y_reg)
plt.show()
sns.scatterplot(x=X["CGPA"], y=y_reg)
plt.show()
sns.boxplot(x=X["Having_Laptop"], y=y_reg)
plt.show()
```

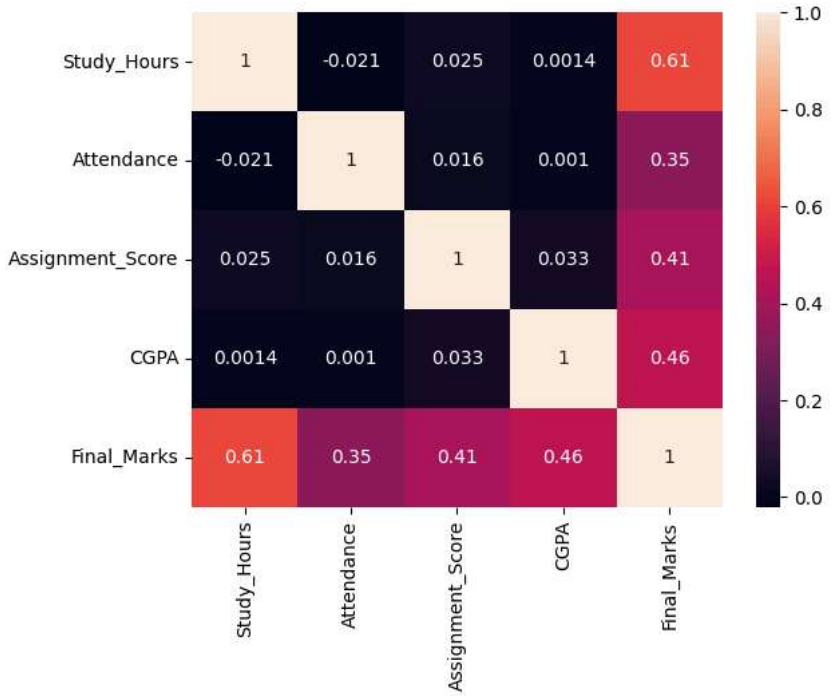






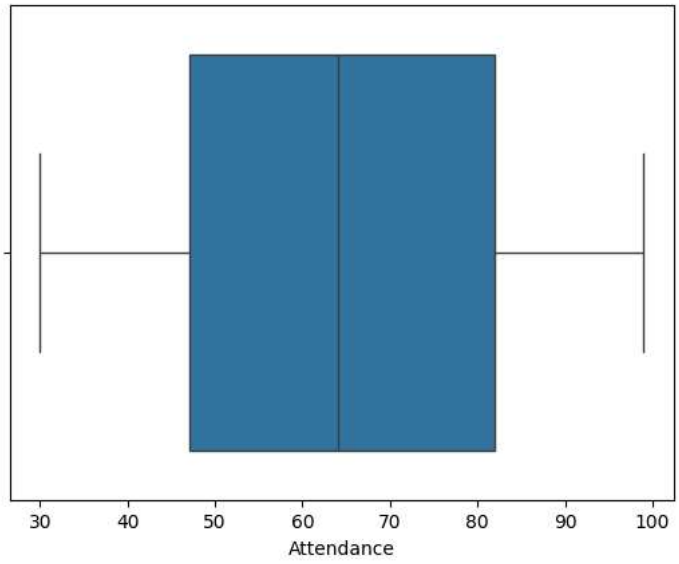
Part F – Correlation Analysis

```
corr = df.drop(["Final_Result"],
axis=1).corr(numeric_only=True)
sns.heatmap(corr, annot=True)
plt.show()
```



Part G – Outlier Detection

```
sns.boxplot(x=X["Attendance"])
plt.show()
```



LAB : 07

DATE : 08/04/2026

DAY : WEDNESDAY

Part A – Dataset Loading & Separation

```
import pandas as pd
url = "https://raw.githubusercontent.com/anshuman-scse/Final_marks_result_prediction/refs/heads/main/Final_marks_result_predict
df = pd.read_csv(url)
X = df.drop(["Final_Marks","Final_Result"], axis=1)
y_reg = df["Final_Marks"]
y_class = df["Final_Result"]
```

Part B – Data Cleaning (handling missing values)

```
#check missing Values
print(X.isnull().sum())
#Fill missing CGPA with mean
X["CGPA"] = X["CGPA"].fillna(X["CGPA"].mean())
```

```
Study_Hours         243
Attendance          227
Assignment_Score    239
Participation       254
Having_Laptop       242
CGPA                243
dtype: int64
```

Part c – Encoding Categorical variable

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X["Participation"] = le.fit_transform(X["Participation"])
X["Having_Laptop"] = le.fit_transform(X["Having_Laptop"])
```

Part D – Feature Scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
numerical_cols = ["Study_Hours","Attendance","Assignment_Score","CGPA"]
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])
```

Part E – feature engineering

```
#Create New Feature
X["performance_Index"] = (X["Study_Hours"] + X["CGPA"])/2
```

Part F – Feature Selection

```
import pandas as pd
# Combine for correlation
temp_df = X.copy()
temp_df["Final_Marks"] = y_reg
corr = temp_df.corr()
print(corr["Final_Marks"].sort_values(ascending=False))
```

```
Final_Marks          1.000000
performance_Index    0.746339
Study_Hours          0.609263
CGPA                 0.449321
Assignment_Score     0.409633
Attendance           0.347170
Having_Laptop        0.002100
Participation       -0.022042
Name: Final_Marks, dtype: float64
```

Part G – Train-Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X, y_reg, test_size=0.2, random_state=42
)
```

LAB : 08

DATE : //2026

DAY : WEDNESDAY/skip technical seminar 1

LAB : 09

DATE : 15/04/2026

DAY : WEDNESDAY

```
import pandas as pd
url = "https://raw.githubusercontent.com/anshuman-scse/Final_marks_result_prediction/refs/heads/main/Final_marks_result_predict
df = pd.read_csv(url)
```

```
print(df.head())
```

```
   Study_Hours  Attendance  Assignment_Score Participation Having_Laptop  \
0          4.0        34.0               2.0        Medium            No
1          9.0        79.0              10.0        Medium           Yes
2          6.0        80.0               5.0        Medium           Yes
3          3.0        53.0               5.0          High           Yes
4          3.0        67.0               0.0          High           Yes

   CGPA  Final_Marks Final_Result
0  6.95           62         Pass
1  8.42          100         Pass
2   NaN           81         Pass
3  3.79           54         Pass
4  7.16           53         Pass
```

```
print(df.isnull().sum())
```

```
Study_Hours         243
Attendance          227
Assignment_Score    239
Participation       254
Having_Laptop       242
CGPA                243
Final_Marks           0
Final_Result          0
dtype: int64
```

```
df = df.dropna(subset = ["CGPA"])
```

```
print(df.isnull().sum())
```

```
Study_Hours         233
Attendance          216
Assignment_Score    229
Participation       239
Having_Laptop       227
CGPA                  0
Final_Marks           0
Final_Result          0
dtype: int64
```

```
df["Study_Hours"] = df["Study_Hours"].fillna(df["Study_Hours"].median())
df["Attendance"] = df["Attendance"].fillna(method="ffill")
df["Assignment_Score"] = df["Assignment_Score"].fillna(df["Assignment_Score"].mean())
```

```
/tmp/ipykernel_1247/859671810.py:2: FutureWarning: Series.fillna with 'method' is deprecated and will raise in a future version.
  df["Attendance"] = df["Attendance"].fillna(method="ffill")
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df["Participation"] = le.fit_transform(df["Participation"])
df["Having_Laptop"] = le.fit_transform(df["Having_Laptop"])
```

```
print(df.isnull().sum())
```

```
Study_Hours         0
Attendance          0
Assignment_Score    0
Participation       0
Having_Laptop       0
CGPA                0
```

```
Final_Marks         0
Final_Result        0
dtype: int64
```

```
df["Performance_Index"] = (df["Study_Hours"] + df["CGPA"]) / 2
```

```
df["Final_Result"] = df["Final_Result"].map({"Pass": 1, "Fail": 0})
```

```
print(df.head())
```

```
   Study_Hours  Attendance  Assignment_Score  Participation  Having_Laptop  \
0          4.0        34.0               2.0              2              0
1          9.0        79.0              10.0              2              1
3          3.0        53.0               5.0              0              1
4          3.0        67.0               0.0              0              1
5          7.0        55.0               4.0              1              1

   CGPA  Final_Marks  Final_Result  Performance_Index
0  6.95           62             1              5.475
1  8.42          100             1              8.710
3  3.79           54             1              3.395
4  7.16           53             1              5.080
5  8.84           78             1              7.920
```

```
print(df.describe())
```

```
       Study_Hours   Attendance  Assignment_Score  Participation  \
count  4757.000000  4757.000000       4757.000000    4757.000000
mean      5.455960    64.182258          4.978799       1.392684
std       3.363962    20.169034          3.101926       0.839328
min       0.000000    30.000000          0.000000       0.000000
25%       3.000000    47.000000          2.000000       1.000000
50%       5.000000    64.000000          5.000000       2.000000
75%       8.000000    82.000000          8.000000       2.000000
max      11.000000    99.000000         10.000000       3.000000

       Having_Laptop         CGPA  Final_Marks  Final_Result  \
count    4757.000000  4757.000000  4757.000000   4757.000000
mean        0.852218     6.524564    69.557494      0.888165
std         0.470561     1.988398    16.269670      0.315197
min         0.000000     3.160000    30.000000      0.000000
25%         1.000000     4.740000    58.000000      1.000000
50%         1.000000     6.530000    69.000000      1.000000
75%         1.000000     8.210000    81.000000      1.000000
max         2.000000     9.890000   100.000000      1.000000

       Performance_Index
count        4757.000000
mean            5.990262
std             1.955583
min             1.580000
25%             4.555000
50%             5.975000
75%             7.420000
max            10.445000
```

```
X = df.drop(["Final_Marks", "Final_Result"], axis=1)
y_reg = df["Final_Marks"]
y_class = df["Final_Result"]
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
numerical_cols = ["Study_Hours","Attendance","Assignment_Score","CGPA"]
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])
```

```
print(X.describe())
```

```
        Study_Hours    Attendance  Assignment_Score  Participation  \
count  4.757000e+03  4.757000e+03      4.757000e+03    4757.000000
mean   7.897824e-17  7.319023e-17      4.929138e-17       1.392684
std    1.000105e+00  1.000105e+00      1.000105e+00       0.839328
min   -1.622056e+00 -1.694967e+00     -1.605236e+00       0.000000
25%   -7.301563e-01 -8.520023e-01     -9.604071e-01       1.000000
50%   -1.355567e-01 -9.037462e-03      6.835638e-03       2.000000
75%    7.563427e-01  8.835136e-01      9.740784e-01       2.000000
max    1.648242e+00  1.726478e+00      1.618907e+00       3.000000

       Having_Laptop          CGPA  Performance_Index
```

```
count    4757.000000  4.757000e+03        4757.000000
mean        0.852218 -3.420523e-16           5.990262
std         0.470561  1.000105e+00           1.955583
min         0.000000 -1.692275e+00           1.580000
25%         1.000000 -8.975825e-01           4.555000
50%         1.000000  2.734246e-03           5.975000
75%         1.000000  8.477242e-01           7.420000
max         2.000000  1.692714e+00          10.445000
```

```
from sklearn.model_selection import train_test_split

X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X, y_reg, test_size=0.2, random_state=42
)
```

```
from sklearn.linear_model import LinearRegression

reg_model = LinearRegression()
reg_model.fit(X_train_reg, y_train_reg)

y_pred_reg = reg_model.predict(X_test_reg)
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(y_test_reg, y_pred_reg)
mse = mean_squared_error(y_test_reg, y_pred_reg)
rmse = np.sqrt(mse)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
```

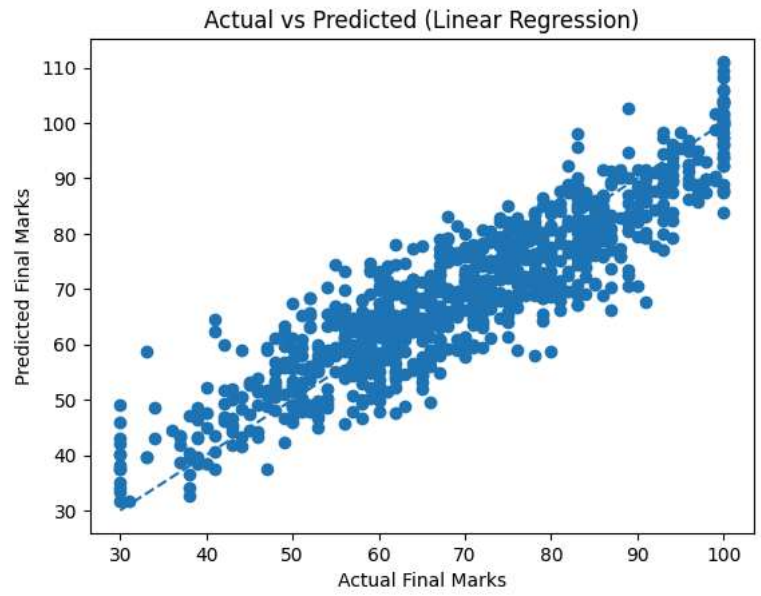
```
MAE: 5.598088098122346
MSE: 49.45657727419057
RMSE: 7.032537043925938
```

```
import matplotlib.pyplot as plt

plt.scatter(y_test_reg, y_pred_reg)
plt.xlabel("Actual Final Marks")
plt.ylabel("Predicted Final Marks")
plt.title("Actual vs Predicted (Linear Regression)")

# Ideal line
plt.plot([y_test_reg.min(), y_test_reg.max()],
         [y_test_reg.min(), y_test_reg.max()],
         linestyle='--')

plt.show()
```



LAB : 10

DATE : 29/04/2026

DAY : WEDNESDAY

Part A – Train-Test Split

```
from sklearn.model_selection import train_test_split
# Regression
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
X, y_reg, test_size=0.2, random_state=42
)
# Classification
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
X, y_class, test_size=0.2, random_state=42
)
```

Part B – Regression (Linear Regression)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
reg_model = LinearRegression()
reg_model.fit(X_train_reg, y_train_reg)
y_pred_reg = reg_model.predict(X_test_reg)
print("MAE:", mean_absolute_error(y_test_reg, y_pred_reg))
print("RMSE:", np.sqrt(mean_squared_error(y_test_reg, y_pred_reg)))
```

```
MAE: 5.598088098122346
RMSE: 7.032537043925938
```

Part C – Classification Models

C1 – Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_cls, y_train_cls)
y_pred_log = log_model.predict(X_test_cls)
acc_log = accuracy_score(y_test_cls, y_pred_log)
print(acc_log)
```

```
0.9369747899159664
```

C2 – Naïve Bayes

```
from sklearn.naive_bayes import GaussianNB
nb_model = GaussianNB()
nb_model.fit(X_train_cls, y_train_cls)
y_pred_nb = nb_model.predict(X_test_cls)
acc_nb = accuracy_score(y_test_cls, y_pred_nb)
print(acc_nb)
```

```
0.9023109243697479
```

C3 – k-NN

```
from sklearn.neighbors import KNeighborsClassifier
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_cls, y_train_cls)
y_pred_knn = knn_model.predict(X_test_cls)
acc_knn = accuracy_score(y_test_cls, y_pred_knn)
print(acc_knn)
```

```
0.9285714285714286
```

C4 – Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_cls, y_train_cls)
y_pred_dt = dt_model.predict(X_test_cls)
acc_dt = accuracy_score(y_test_cls, y_pred_dt)
print(acc_dt)
```

```
0.9023109243697479
```

C5 – Random Forest

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_cls, y_train_cls)
y_pred_rf = rf_model.predict(X_test_cls)
acc_rf = accuracy_score(y_test_cls, y_pred_rf)
print(acc_rf)
```

```
0.9296218487394958
```

C6 – Support Vector Machine (SVM)

```
from sklearn.svm import SVC
svm_model = SVC(kernel='linear')
svm_model.fit(X_train_cls, y_train_cls)
y_pred_svm = svm_model.predict(X_test_cls)
acc_svm = accuracy_score(y_test_cls, y_pred_svm)
print(acc_svm)
#
```

```
0.9327731092436975
```

Part D – Model Comparison

```
import pandas as pd

results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Naive Bayes",
        "k-NN",
        "Decision Tree",
        "Random Forest",
        "SVM"
    ],
    "Accuracy": [
        acc_log,
        acc_nb,
        acc_knn,
        acc_dt,
        acc_rf,
        acc_svm
    ]
})

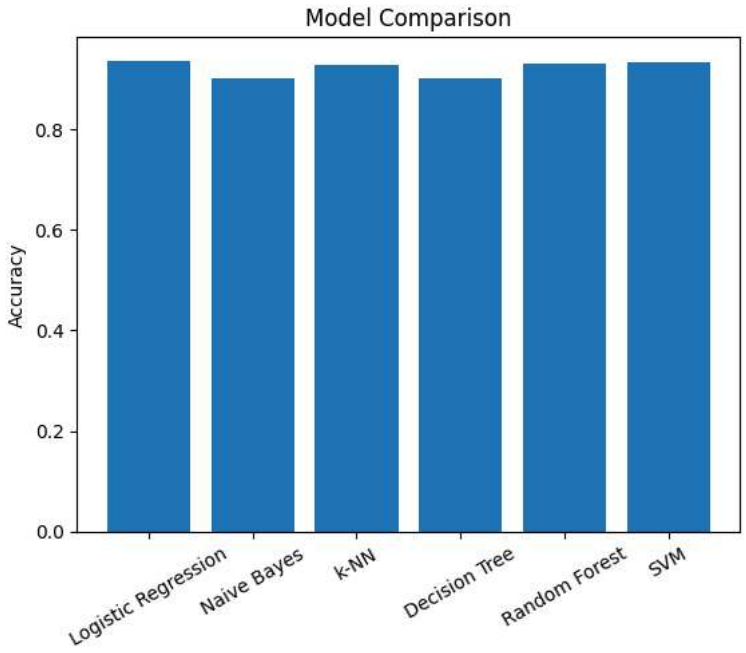
print(results)
```

```
                Model  Accuracy
0  Logistic Regression  0.936975
1          Naive Bayes  0.902311
2                k-NN  0.928571
3        Decision Tree  0.902311
4        Random Forest  0.929622
5                  SVM  0.932773
```

Visualization – Model Comparison

```
import matplotlib.pyplot as plt

plt.bar(results["Model"], results["Accuracy"])
plt.xticks(rotation=30)
plt.ylabel("Accuracy")
plt.title("Model Comparison")
plt.show()
```

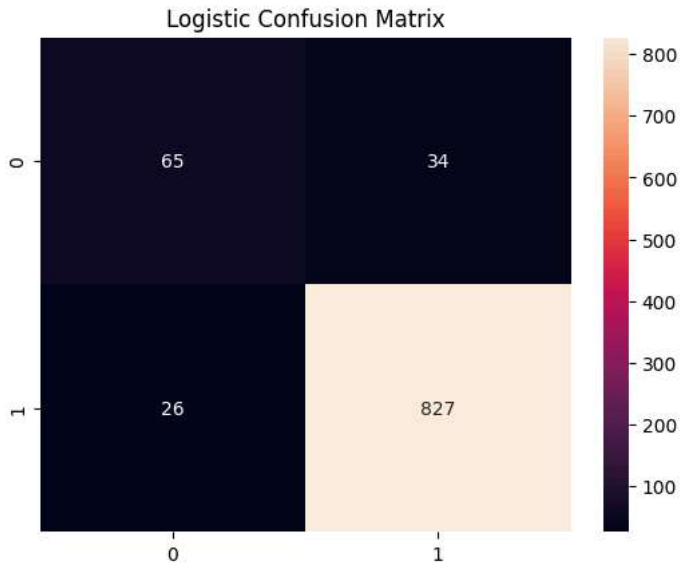


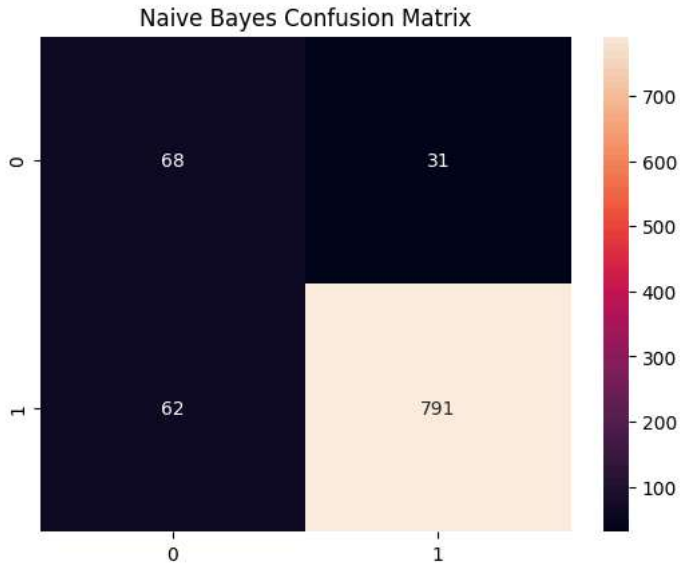
Part E – Confusion Matrix (All Models)

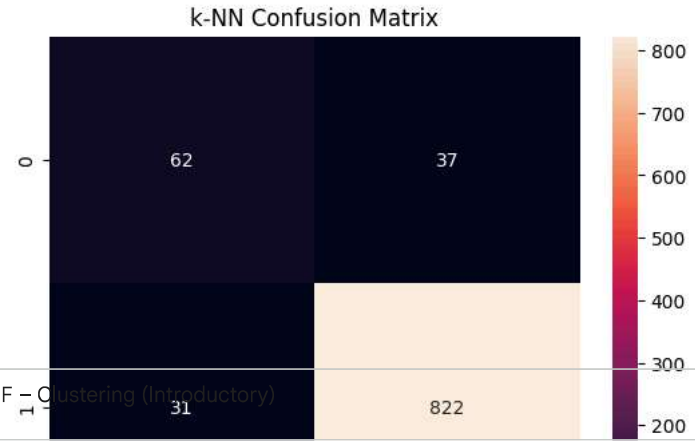
```
import seaborn as sns
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

predictions = {
    "Logistic": y_pred_log,
    "Naive Bayes": y_pred_nb,
    "k-NN": y_pred_knn,
    "Decision Tree": y_pred_dt,
    "Random Forest": y_pred_rf,
    "SVM": y_pred_svm
}

for name, pred in predictions.items():
    cm = confusion_matrix(y_test_cls, pred)
    sns.heatmap(cm, annot=True, fmt='d')
    plt.title(f"{name} Confusion Matrix")
    plt.show()
```







Part F – Clustering (Introductory)

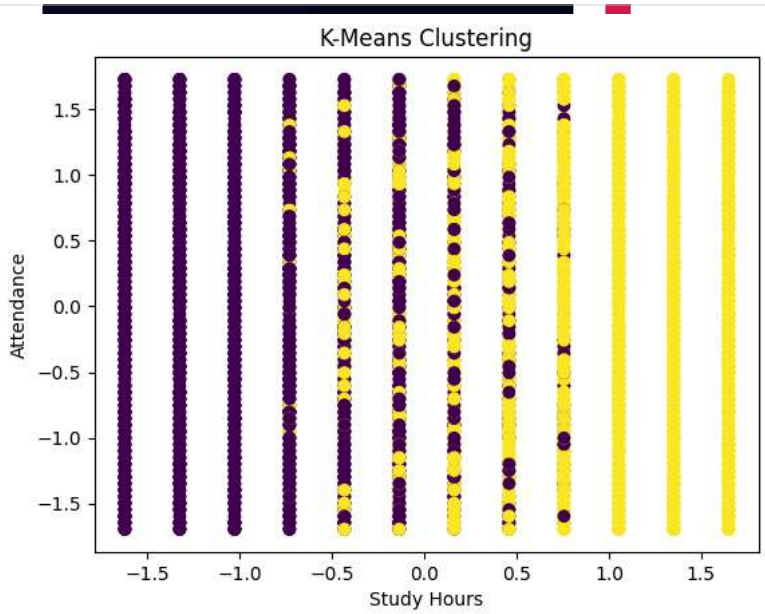
```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2, random_state=42)
clusters = kmeans.fit_predict(X)
X["Cluster"] = clusters
```

Cluster Visualization




```
plt.scatter(X["Study_Hours"], X["Attendance"], c=X["Cluster"])
plt.xlabel("Study Hours")
plt.ylabel("Attendance")
plt.title("K-Means Clustering")
```

```
plt.show()
```



Additional Lab Manual: Handling dataset imbalance

Step 1 – Check Class Distribution

```
print(df["Final_Result"].value_counts())
```

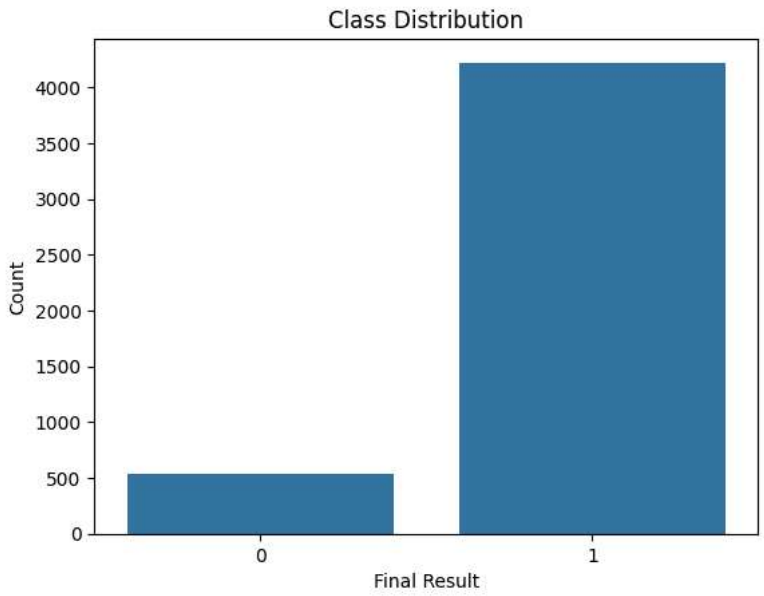
```
Final_Result
1    4225
0     532
Name: count, dtype: int64
```

Step 2 – Visualize Imbalance




```
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x=df["Final_Result"])
plt.title("Class Distribution")
plt.xlabel("Final Result")
plt.ylabel("Count")
plt.show()
```





Option A – Random Oversampling (Simple & Beginner Friendly)

A1 – Separate Classes

```
pass_data = df[df["Final_Result"] == 1]
fail_data = df[df["Final_Result"] == 0]
```

A2 – Oversample Minority Class

```
from sklearn.utils import resample
fail_oversampled = resample(
fail_data,
replace=True,
n_samples=len(pass_data),
random_state=42
)
df_balanced = pd.concat([pass_data, fail_oversampled])
```

A3 – Verify Balanced Dataset

```
print(df_balanced["Final_Result"].value_counts())
```

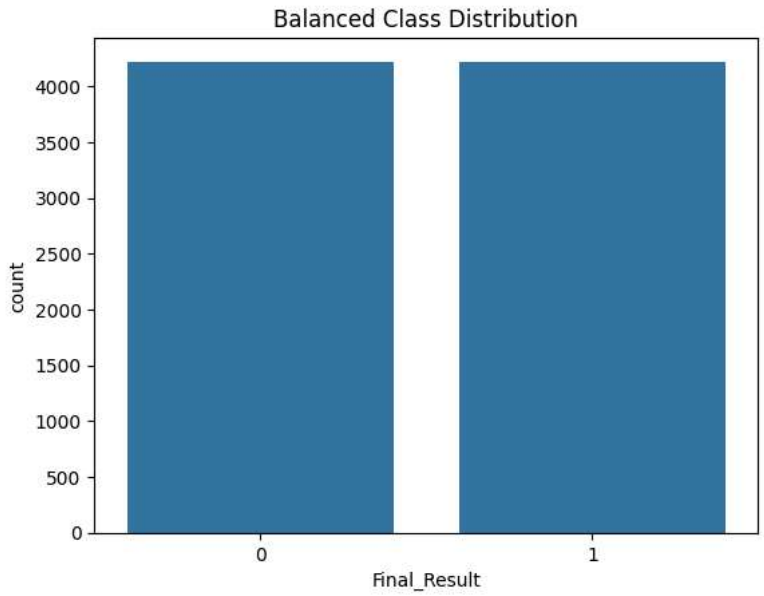
```
Final_Result
1    4225
0    4225
Name: count, dtype: int64
```

```
df_balanced.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 8450 entries, 0 to 4203
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Study_Hours        8450 non-null   float64
 1   Attendance         8450 non-null   float64
 2   Assignment_Score   8450 non-null   float64
 3   Participation      8450 non-null   int64
 4   Having_Laptop      8450 non-null   int64
 5   CGPA               8450 non-null   float64
 6   Final_Marks        8450 non-null   int64
 7   Final_Result       8450 non-null   int64
 8   Performance_Index  8450 non-null   float64
dtypes: float64(5), int64(4)
memory usage: 660.2 KB
```

A4 – Visualization

```
sns.countplot(x=df_balanced["Final_Result"])
plt.title("Balanced Class Distribution")
plt.show()
```



Option B – Random Undersampling

```
pass_undersampled = resample(
pass_data,
replace=False,
n_samples=len(fail_data),
random_state=42
)
df_balanced = pd.concat([pass_undersampled, fail_data])
```

Option C – SMOTE (Most Important for ML) SMOTE = Synthetic Minority Oversampling Technique Instead of duplicating records, it creates synthetic new samples.

C1 – Install imbalanced-learn

```
!pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dist-packages (0.14.1)
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (2.0.2)
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.16.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.6.1)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (0.
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.5.3)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (3.6.0
```

C2 – Apply SMOTE

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y_class)
```

C3 – Check New Distribution

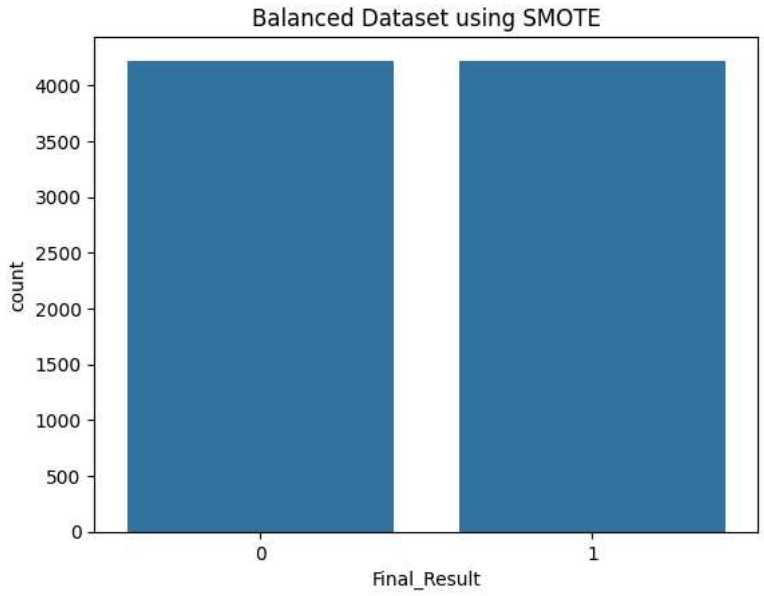
```
print(pd.Series(y_resampled).value_counts())
```

```
Final_Result
1    4225
```

```
0    4225
Name: count, dtype: int64
```

C4 – Visualization

```
sns.countplot(x=y_resampled)
plt.title("Balanced Dataset using SMOTE")
plt.show()
```



Lab – 11

Day:- Wednesday

Date:- 13/05/2026

```
import pandas as pd
url = "https://raw.githubusercontent.com/anshuman-scse/Final_marks_result_prediction/refs/heads/main/Final_marks_result_predict
df = pd.read_csv(url)
```

4. Feature Selection for Clustering

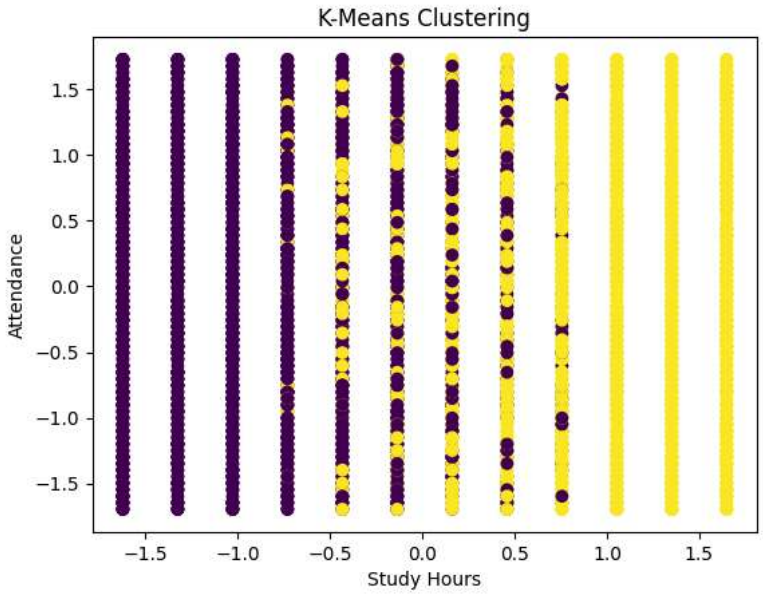
```
X_cluster = X.copy()
```

Part A – K-Means Clustering

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans_labels = kmeans.fit_predict(X_cluster)
X_cluster["KMeans_Cluster"] = kmeans_labels
```

A2 – Visualization

```
import matplotlib.pyplot as plt
plt.scatter(
X_cluster["Study_Hours"],
X_cluster["Attendance"],
c=X_cluster["KMeans_Cluster"]
)
plt.xlabel("Study Hours")
plt.ylabel("Attendance")
plt.title("K-Means Clustering")
plt.show()
```



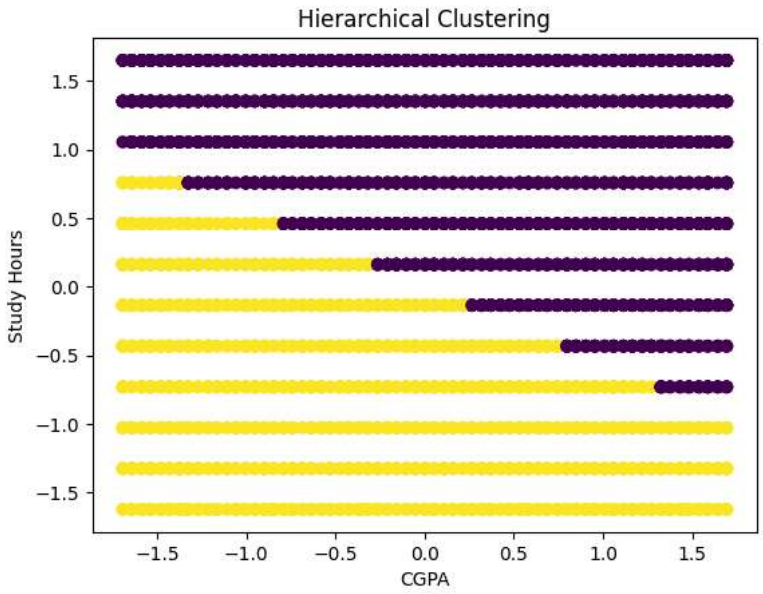
Part B – Hierarchical Clustering

B1 – Apply Hierarchical Clustering

```
from sklearn.cluster import AgglomerativeClustering
hc = AgglomerativeClustering(n_clusters=2)
hc_labels = hc.fit_predict(X_cluster)
X_cluster["HC_Cluster"] = hc_labels
```

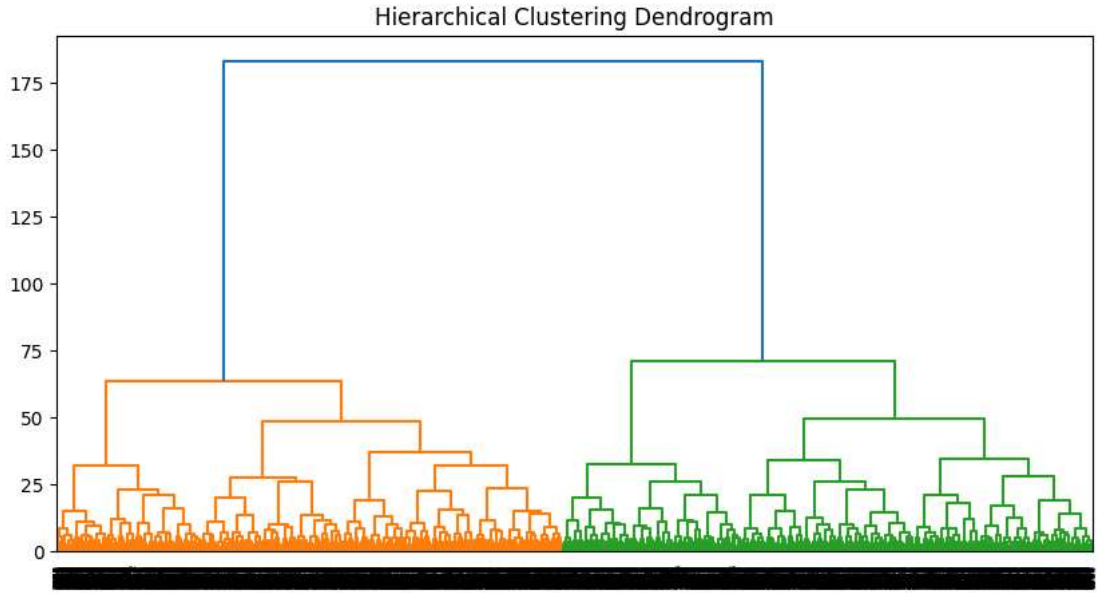
B2 – Visualization

```
plt.scatter(
X_cluster["CGPA"],
X_cluster["Study_Hours"],
c=X_cluster["HC_Cluster"]
)
plt.xlabel("CGPA")
plt.ylabel("Study Hours")
plt.title("Hierarchical Clustering")
plt.show()
```



B3 – Dendrogram Visualization

```
from scipy.cluster.hierarchy import dendrogram, linkage
linked = linkage(X_cluster.drop(["KMeans_Cluster","HC_Cluster"], axis=1),
method='ward')
plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Hierarchical Clustering Dendrogram")
plt.show()
```



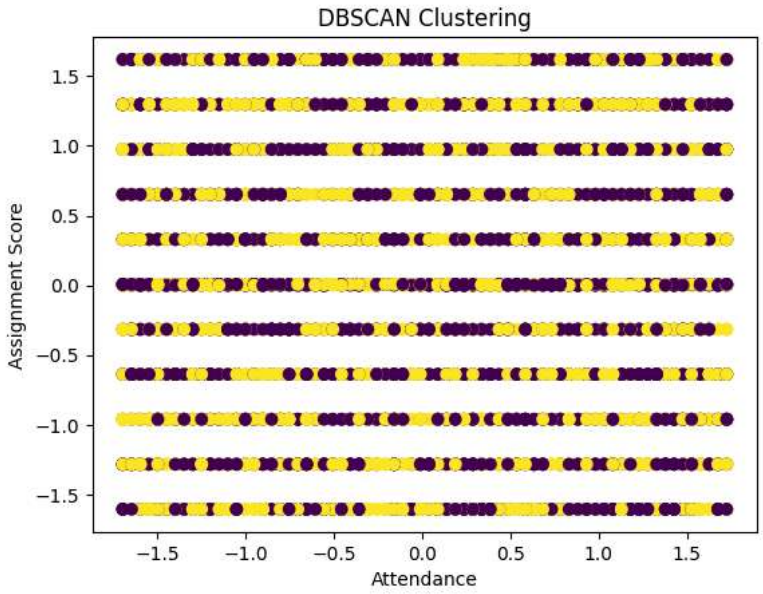
Part C – DBSCAN Clustering

C1 – Apply DBSCAN

```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=1.5, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_cluster)
X_cluster["DBSCAN_Cluster"] = dbscan_labels
```

C2 – Visualization

```
plt.scatter(
X_cluster["Attendance"],
X_cluster["Assignment_Score"],
c=X_cluster["DBSCAN_Cluster"]
)
plt.xlabel("Attendance")
plt.ylabel("Assignment Score")
plt.title("DBSCAN Clustering")
plt.show()
```



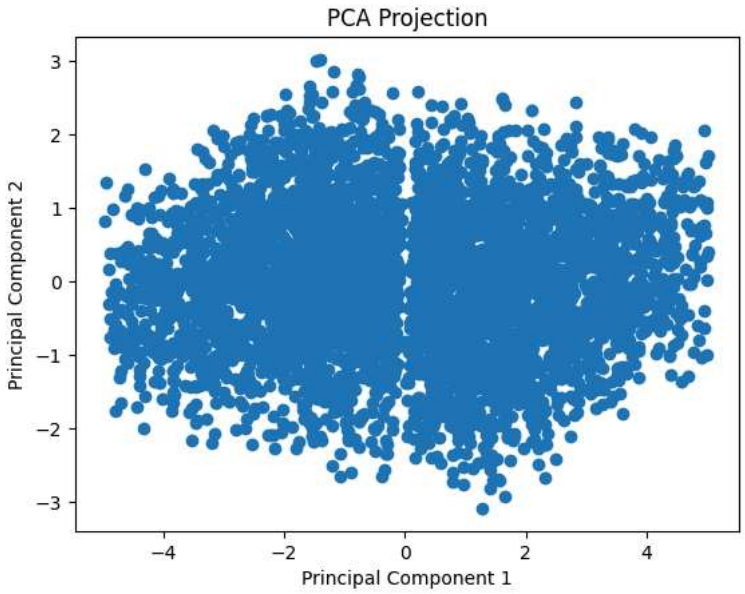
Part D – PCA (Principal Component Analysis)

D1 – Apply PCA

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_cluster.drop(
["KMeans_Cluster","HC_Cluster","DBSCAN_Cluster"],
axis=1
))
```

D2 – Visualization

```
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Projection")
plt.show()
```



D3 – Explained Variance

```
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

```
Explained Variance Ratio:
[0.55592264 0.11385678]
```

Part E – Cross Validation

E1 – Logistic Regression Cross Validation

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
scores = cross_val_score(
model,
X,
y_class,
cv=5,
scoring='accuracy'
)
print("Cross Validation Scores:", scores)
print("Average Accuracy:", scores.mean())
```

```
Cross Validation Scores: [0.93592437 0.93802521 0.92744479 0.93585699 0.92954784]
Average Accuracy: 0.9333598423596567
```

Part F – Evaluation Metrics

F1 – Classification Metrics

```
from sklearn.metrics import classification_report
model.fit(X_train_cls, y_train_cls)
pred = model.predict(X_test_cls)
print(classification_report(y_test_cls, pred))
```

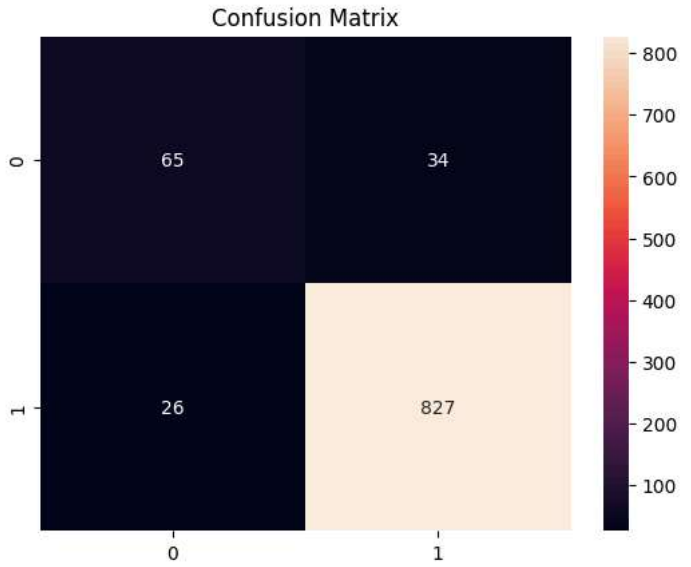
```
              precision    recall  f1-score   support

           0       0.71      0.66      0.68        99
           1       0.96      0.97      0.96       853

    accuracy                           0.94       952
   macro avg       0.84      0.81      0.82       952
weighted avg       0.93      0.94      0.94       952
```

F2 – Confusion Matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
cm = confusion_matrix(y_test_cls, pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()
```



Part A – Train-Test Split

```
from sklearn.model_selection import train_test_split
# Regression
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
X, y_reg, test_size=0.2, random_state=42
)
# Classification
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
X, y_class, test_size=0.2, random_state=42
)
```

Part B – Regression Evaluation

B1 – Train Linear Regression Model

```
from sklearn.linear_model import LinearRegression
reg_model = LinearRegression()
reg_model.fit(X_train_reg, y_train_reg)
y_pred_reg = reg_model.predict(X_test_reg)
```

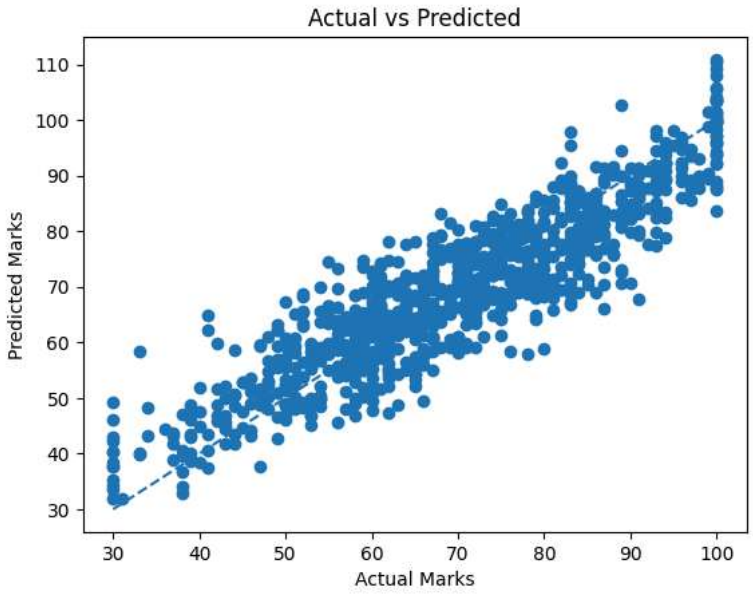
B2 – Regression Metrics

```
from sklearn.metrics import (
mean_absolute_error,
mean_squared_error,
r2_score
)
import numpy as np
mae = mean_absolute_error(y_test_reg, y_pred_reg)
mse = mean_squared_error(y_test_reg, y_pred_reg)
rmse = np.sqrt(mse)
r2 = r2_score(y_test_reg, y_pred_reg)
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MAE: 5.5935804426901266
MSE: 49.50372420630687
RMSE: 7.035888302574656
R2 Score: 0.8103084126401183
```

B3 – Actual vs Predicted Visualization

```
import matplotlib.pyplot as plt
plt.scatter(y_test_reg, y_pred_reg)
plt.xlabel("Actual Marks")
plt.ylabel("Predicted Marks")
plt.title("Actual vs Predicted")
plt.plot(
[y_test_reg.min(), y_test_reg.max()],
[y_test_reg.min(), y_test_reg.max()],
linestyle='--'
)
plt.show()
```



Part C – Classification Evaluation

C1 – Train Logistic Regression Model

```
from sklearn.linear_model import LogisticRegression
cls_model = LogisticRegression(max_iter=1000)
cls_model.fit(X_train_cls, y_train_cls)
y_pred_cls = cls_model.predict(X_test_cls)
```

C2 – Accuracy Score

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9369747899159664
```

C3 – Classification Report

```
from sklearn.metrics import classification_report
print(classification_report(y_test_cls, y_pred_cls))
```

```
              precision    recall  f1-score   support

           0       0.71      0.66      0.68        99
           1       0.96      0.97      0.96       853

    accuracy                           0.94       952
   macro avg       0.84      0.81      0.82       952
weighted avg       0.93      0.94      0.94       952
```

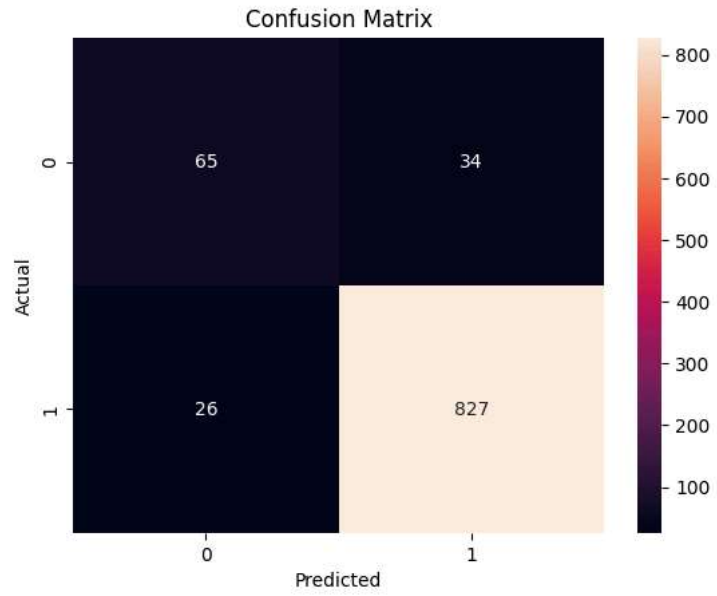
C4 – Confusion Matrix

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test_cls, y_pred_cls)
print(cm)
```

```
[[ 65  34]
 [ 26 827]]
```

C5 – Confusion Matrix Visualization

```
import seaborn as sns
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



D1 – Apply Cross Validation

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(
cls_model,
X,
y_class,
cv=5,
scoring='accuracy'
)
print("Cross Validation Scores:", scores)
```

```
Cross Validation Scores: [0.93592437 0.93802521 0.92744479 0.93585699 0.92954784]
```

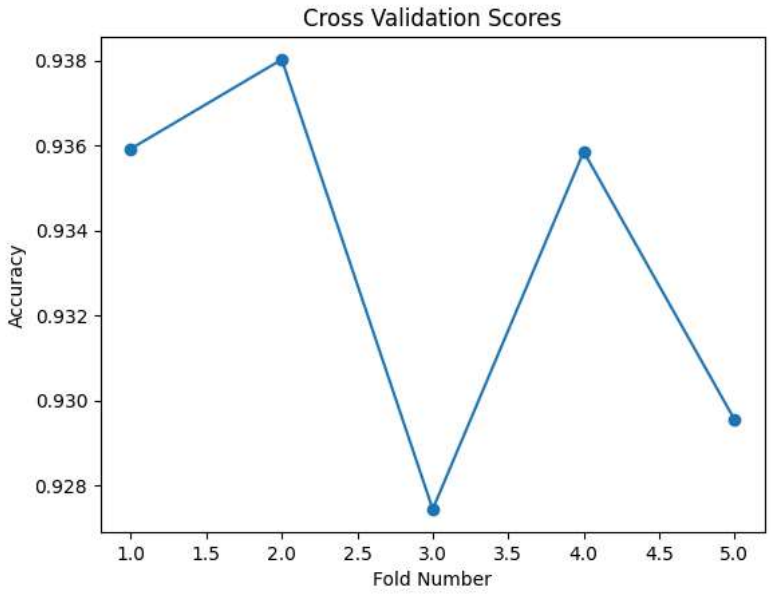
D2 – Average Cross Validation Score

```
print("Average Accuracy:", scores.mean())
```

```
Average Accuracy: 0.9333598423596567
```

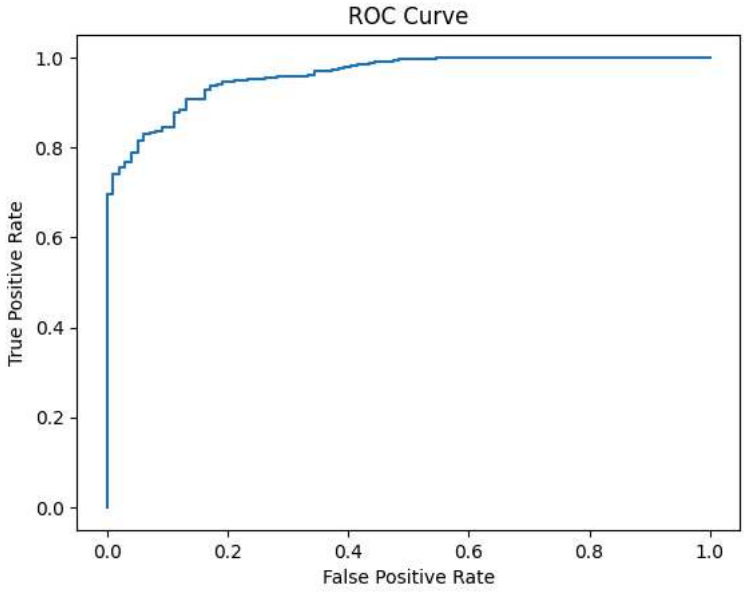
D3 – Visualization of Fold Scores

```
plt.plot(range(1,6), scores, marker='o')
plt.xlabel("Fold Number")
plt.ylabel("Accuracy")
plt.title("Cross Validation Scores")
plt.show()
```



Part E – ROC Curve (Optional Advanced Section)

```
from sklearn.metrics import roc_curve, auc
y_prob = cls_model.predict_proba(X_test_cls)[:,1]
fpr, tpr, thresholds = roc_curve(y_test_cls, y_prob)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()
print("AUC:", roc_auc)
```




```
AUC: 0.9604248818785748
```

Q1.can over sampling lead to data leakage ? Yes. If you oversample before splitting your data, exact duplicates or highly similar synthetic points will end up in both the training and testing sets. This causes data leakage, giving you artificially high test scores while the model fails on real-world data.

Rule: Always split your data into train/test sets first, then oversample only the training data.

Q2.what would happen if the cross validationis done before train - test split? You lose your only unbiased way to test the model.

If you run cross-validation on the entire dataset first, every data point is used to tune and select the model. Without a completely isolated, untouched test set held out from the very beginning, you cannot truly know if your model has overfit or how it will perform on genuinely

new data.

Correct Workflow: > 1. Split into Train and Test sets. 2. Run cross-validation only on the Train set. 3. Do a final evaluation on the untouched Test set.

Lab – 12

Day:- Wednesday

Date:- 20/05/2026

Part A – Train-Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
X,
y_class,
test_size=0.2,
random_state=42
)
```

Part B – Import Neural Network Libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Part C – Build Neural Network Model

C1 – Create ANN Architecture

```
model = Sequential()
# Input + Hidden Layer 1
model.add(Dense(16, activation='relu', input_shape=(X.shape[1],)))
# Hidden Layer 2
model.add(Dense(8, activation='relu'))
# Output Layer
model.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Part D – Compile Model

```
model.compile(
optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy']
)
```

Part E – Train Neural Network

```
history = model.fit(
X_train,
y_train,
epochs=50,
batch_size=8,
validation_split=0.2
)
```

```
Epoch 26/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9369 - loss: 0.1490 - val_accuracy: 0.9198 - val_loss: 0.1913
Epoch 27/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9366 - loss: 0.1477 - val_accuracy: 0.9172 - val_loss: 0.1933
Epoch 28/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9386 - loss: 0.1480 - val_accuracy: 0.9198 - val_loss: 0.1910
Epoch 29/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9350 - loss: 0.1479 - val_accuracy: 0.9198 - val_loss: 0.1910
Epoch 30/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9382 - loss: 0.1480 - val_accuracy: 0.9172 - val_loss: 0.1919
Epoch 31/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9389 - loss: 0.1466 - val_accuracy: 0.9185 - val_loss: 0.1918
Epoch 32/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9386 - loss: 0.1468 - val_accuracy: 0.9159 - val_loss: 0.1929
Epoch 33/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9369 - loss: 0.1467 - val_accuracy: 0.9159 - val_loss: 0.1931
Epoch 34/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9379 - loss: 0.1467 - val_accuracy: 0.9172 - val_loss: 0.1948
Epoch 35/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9373 - loss: 0.1468 - val_accuracy: 0.9185 - val_loss: 0.1966
Epoch 36/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9373 - loss: 0.1452 - val_accuracy: 0.9146 - val_loss: 0.1931
Epoch 37/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9369 - loss: 0.1461 - val_accuracy: 0.9172 - val_loss: 0.1943
Epoch 38/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9382 - loss: 0.1461 - val_accuracy: 0.9159 - val_loss: 0.1922
Epoch 39/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9369 - loss: 0.1449 - val_accuracy: 0.9172 - val_loss: 0.1956
Epoch 40/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9392 - loss: 0.1451 - val_accuracy: 0.9159 - val_loss: 0.1961
Epoch 41/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9396 - loss: 0.1449 - val_accuracy: 0.9185 - val_loss: 0.1944
Epoch 42/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9376 - loss: 0.1463 - val_accuracy: 0.9198 - val_loss: 0.1913
Epoch 43/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9389 - loss: 0.1460 - val_accuracy: 0.9146 - val_loss: 0.1941
Epoch 44/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9379 - loss: 0.1448 - val_accuracy: 0.9172 - val_loss: 0.1958
Epoch 45/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9389 - loss: 0.1448 - val_accuracy: 0.9159 - val_loss: 0.1971
Epoch 46/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9376 - loss: 0.1440 - val_accuracy: 0.9146 - val_loss: 0.1944
Epoch 47/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9389 - loss: 0.1444 - val_accuracy: 0.9185 - val_loss: 0.1951
Epoch 48/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9379 - loss: 0.1437 - val_accuracy: 0.9172 - val_loss: 0.1952
Epoch 49/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9376 - loss: 0.1443 - val_accuracy: 0.9159 - val_loss: 0.1949
Epoch 50/50
381/381 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9379 - loss: 0.1437 - val_accuracy: 0.9172 - val_loss: 0.1976
```

Part F – Evaluate Model

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)
print("Test Loss:", loss)
```

```
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9328 - loss: 0.1479
Test Accuracy: 0.9327731132507324
Test Loss: 0.14789703488349915
```

Part G – Predictions

```
predictions = model.predict(X_test)
predicted_classes = (predictions > 0.5).astype(int)
print(predicted_classes[:10])
```

```
30/30 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
[[1]
 [1]
 [1]
 [1]
 [1]
 [1]
 [1]
 [1]
 [1]
 [1]]
```

Part H – Training Accuracy Visualization